# Programación por bloques

Nombre: _____ Fecha: _____

## Qué vas a aprender

- Explicar qué es un programa y un lenguaje de programación
- Reconocer las ventajas de los lenguajes por bloques y algunos entornos
- Identificar los tipos de bloques: eventos, acciones, control, sensores y datos
- Crear, probar y depurar un programa sencillo por bloques

## Para pensar

**Si a un robot le dices «tráeme un vaso de agua», no se mueve. ¿Qué le falta para hacerlo?**

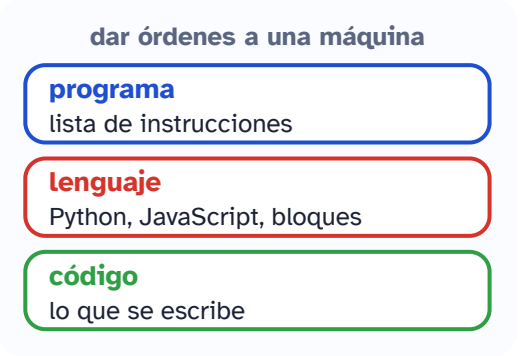
Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

Rechazar Aceptar

## Explicación

### 1.1 ¿Qué es programar?

Un programa es una lista de **instrucciones** ordenadas que le dice a un ordenador o a otra máquina lo que tiene que hacer. Todas las aplicaciones del móvil, los videojuegos, las páginas web, el control de un semáforo o el de una lavadora son programas. Programar es escribir esas instrucciones. Como los ordenadores no entienden nuestro idioma, las instrucciones se escriben en un lenguaje de programación, un lenguaje con unas palabras y unas reglas muy precisas, que luego se traduce a ceros y unos. Hay muchos: Python, JavaScript, C, Java... El texto escrito en ellos se llama **código**. Para programar bien no basta con conocer el lenguaje: hay que saber **pensar el problema**, dividirlo en partes más sencillas y ordenar los pasos, algo que se llama pensamiento computacional. Programar no es solo cosa de informáticos: hoy se usa en ciencia, arte, música, medicina, ingeniería y casi cualquier profesión.




### 1.2 Los algoritmos

Antes de programar hay que pensar **cómo** se resuelve el problema. Un algoritmo es una serie de pasos ordenados y precisos para resolver un problema o hacer una tarea. Los usamos a diario sin darnos cuenta: una receta de cocina, las instrucciones para montar un mueble, el camino de casa al instituto o la forma de hacer una división.

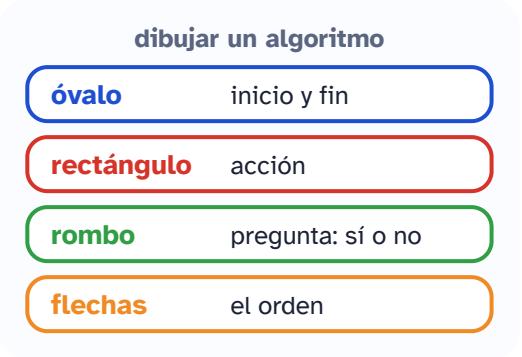
Un buen algoritmo cumple tres condiciones:

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

- Es **finito**: termina en algún momento.

Los algoritmos se pueden escribir con palabras o dibujar en un diagrama de flujo, con formas que tienen un significado: un óvalo para el **inicio** y el **fin**, un rectángulo para cada **acción**, un rombo para las **preguntas** (que tienen dos salidas, sí o no) y flechas que indican el orden. Por ejemplo, para cruzar la calle: inicio → llegar al paso de peatones → ¿el semáforo está en verde? Si no, esperar y volver a preguntar; si sí, mirar a los dos lados y cruzar → fin.

Esta forma de descomponer un problema en pasos, encontrar patrones y pensar en soluciones que pueda ejecutar una máquina se llama **pensamiento computacional**.




## 1.3 Los lenguajes por bloques

Escribir código en un lenguaje de texto es difícil al principio: un punto y coma olvidado o una palabra mal escrita hacen que el programa no funcione. Para aprender, se crearon los lenguajes de programación por bloques, en los que las instrucciones son **piezas de colores** que se arrastran con el ratón y se encajan como un puzle. Tienen muchas ventajas: no hay errores de escritura, porque las instrucciones ya están escritas; los bloques solo encajan donde tiene sentido; los colores indican la categoría de cada bloque; y se ve el resultado al momento. Así uno se puede concentrar en la **lógica** del programa. Los entornos más conocidos son Scratch, para crear historias, animaciones y juegos; MakeCode, para programar placas como la **micro:bit**; **App Inventor**, para crear aplicaciones de móvil; y los editores de robots educativos como mBlock. Lo que se aprende con bloques (secuencias, bucles, condiciones, variables) sirve después para cualquier

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información




las piezas solo encajan donde tienen sentido

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

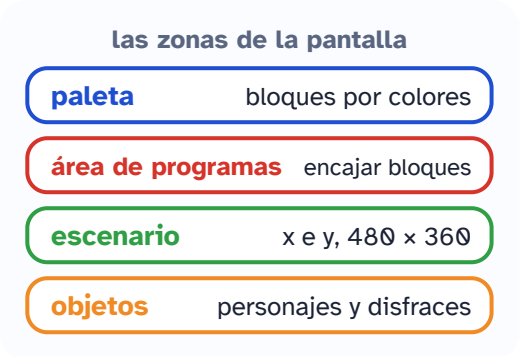
## 1.4 Scratch: escenario, objetos y disfraces

Scratch es el entorno de bloques más usado del mundo. Lo creó el MIT (una universidad de Estados Unidos), es gratuito y funciona en el navegador. Su pantalla tiene cuatro zonas:

- La **paleta de bloques**, a la izquierda, con las categorías de colores.
- El **área de programas**, en el centro, donde se arrastran y encajan los bloques.
- El **escenario**, arriba a la derecha, donde se ve lo que pasa. Mide 480 × 360 pasos; el centro es el punto (0, 0), la **x** indica la posición horizontal y la **y**, la vertical.
- La **lista de objetos**, debajo, con los personajes del proyecto.

Los **objetos** (o sprites) son los personajes y elementos que se mueven: el gato, una pelota, un enemigo. Cada objeto tiene **sus propios programas**. Además, cada objeto puede tener varios **disfraces**, que son dibujos distintos: cambiando rápidamente de disfraz se consigue que el personaje parezca andar. El escenario también puede tener varios **fondos**, por ejemplo uno para cada nivel del juego.

Los proyectos se pueden **compartir** en la comunidad de Scratch, y cualquiera puede ver sus bloques y hacer una versión propia (remezcla), citando siempre al autor original.




## 2.1 Tipos de bloques

En todos los entornos de bloques, las instrucciones se agrupan en **categorías**, cada una con un color. Los bloques de eventos inician el programa cuando

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

acción hacen algo visible: mover, girar, mostrar un dibujo, encender una luz, tocar un sonido, decir un mensaje. Los bloques de control deciden el orden en que se ejecutan las instrucciones: esperar, repetir (bucles) o hacer algo solo si se cumple una condición («si… entonces»). Los bloques sensores obtienen información del entorno: si se toca algo, si se pulsa una tecla, la temperatura, la luz, la posición del ratón. Los bloques de **operadores** hacen cálculos y comparaciones (sumar, mayor que, número al azar). Y los bloques de **datos** o **variables** guardan valores que pueden cambiar, como los puntos de un juego o el tiempo que queda.

| Categoría | Para qué sirve | Ejemplo |
|---|---|---|
| Eventos | empezar cuando pasa algo | al pulsar el botón A |
| Acción | hacer algo visible o audible | mostrar un icono, mover 10 pasos |
| Control | esperar, repetir, decidir | repetir 4 veces, si… entonces |
| Sensores | leer el entorno | temperatura, ¿tocando borde? |
| Operadores | calcular y comparar | número al azar, mayor que |
| Variables | guardar valores que cambian | puntos, vidas |

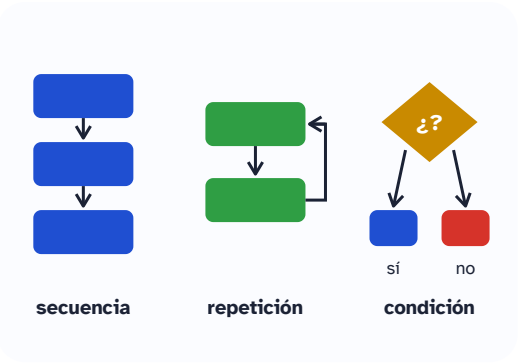



## 2.2 Secuencia, repetición y condición

Casi todos los programas se construyen combinando tres estructuras. La secuencia: las instrucciones se ejecutan **una detrás de otra**, en el orden en que están colocadas, de arriba abajo. El orden importa: no es lo mismo «girar y avanzar» que «avanzar y girar». La repetición, o **bucle**: un grupo de instrucciones

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

repetir 4 veces «avanzar y girar 90°». La condición: un grupo de instrucciones solo se ejecuta **si se cumple algo** («si la temperatura es mayor que 25, mostrar un sol»), y se puede añadir qué hacer en caso contrario («si no, mostrar una nube»). Combinando estas tres estructuras con eventos, sensores y variables se pueden crear programas sorprendentemente complejos, desde un termómetro digital hasta un videojuego.




## 2.3 Variables y operadores

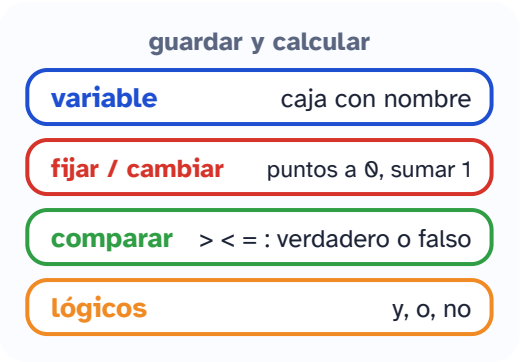
Una variable es como una caja con nombre donde el programa guarda un dato que puede cambiar: los **puntos** de un juego, las **vidas** que quedan, el **tiempo**, la tirada de un dado o el nombre del jugador. Con una variable se pueden hacer tres cosas:

- **Crearla** y darle un nombre que explique lo que guarda.
- **Fijar** su valor: «fijar puntos a 0» al empezar el juego.
- **Cambiar** su valor: «sumar 1 a puntos» cada vez que el gato toca el ratón.

Los operadores son los bloques que hacen cálculos y comparaciones:

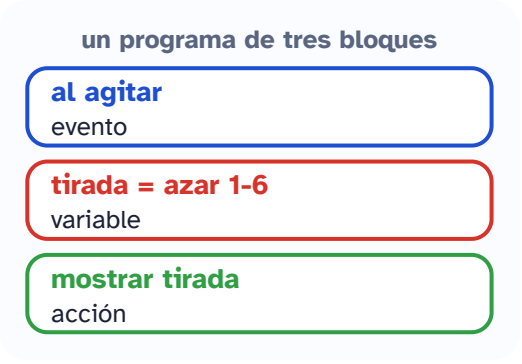
- **Aritméticos**: sumar, restar, multiplicar, dividir.
- **De comparación**: mayor que (>), menor que (<), igual a (=). Su respuesta es **verdadero o falso**, y por eso encajan dentro de las condiciones: «si puntos > 10 entonces cambiar al nivel 2».
- **Lógicos**: **y** (se tienen que cumplir las dos cosas), **o** (basta con una) y **no** (lo contrario). Por ejemplo: «si tocando al enemigo **y** vidas = 1 entonces fin del juego».
- El **número al azar**, para que el juego no sea siempre igual.

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información




## 2.4 Un ejemplo con la placa micro:bit

La micro:bit es una pequeña placa programable del tamaño de una tarjeta, con una matriz de 25 luces LED, dos botones, sensores de luz, temperatura y movimiento, brújula y radio. Se programa gratis desde el navegador con **MakeCode**, que incluye un simulador para probar los programas sin tener la placa. Veamos un **dado electrónico**. Necesitamos: un evento, «al agitar»; un bloque de variables, «fijar *tirada* a número al azar entre 1 y 6»; y un bloque de acción, «mostrar número *tirada*». Al agitar la placa, el sensor de movimiento lo detecta, se elige un número al azar, se guarda en la variable y aparece en las luces. Podemos mejorarlo: con un bloque «si *tirada* = 6 entonces» mostrar además una carita feliz, o con un bucle «repetir 5 veces» mostrar números al azar muy rápido antes del resultado final, para que parezca que el dado rueda. Este pequeño proyecto ya usa eventos, variables, operadores, condiciones y bucles.



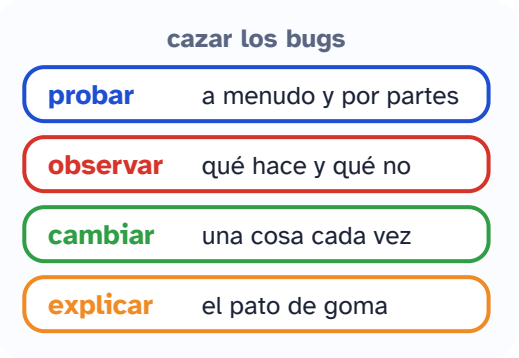

## 3.1 Probar y depurar

Es normalísimo que un programa no funcione bien a la primera: hasta los

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

bloques en un **orden equivocado**; olvidar el **bloque de evento**, de modo que el programa nunca empieza; poner un número incorrecto (girar 9 grados en vez de 90); usar «repetir» en lugar de «por siempre», o al revés; y no **reiniciar** las variables o la posición al empezar. Para depurar conviene: ejecutar el programa **paso a paso**, observando qué hace cada bloque; probar **una parte cada vez**; leer el programa en voz alta, como si fuéramos la máquina; añadir bloques que muestren el valor de las variables; y cambiar **una sola cosa** cada vez para ver qué efecto tiene. Probar con distintos casos, también los raros, ayuda a encontrar errores escondidos.

Una técnica muy útil es la del **pato de goma**: explicar el programa en voz alta, bloque a bloque, a un compañero o incluso a un muñeco. Muchas veces, al explicarlo, uno mismo descubre dónde está el error. Otra es hacer que el personaje **diga** el valor de una variable en cada momento, para ver si cambia como esperábamos.



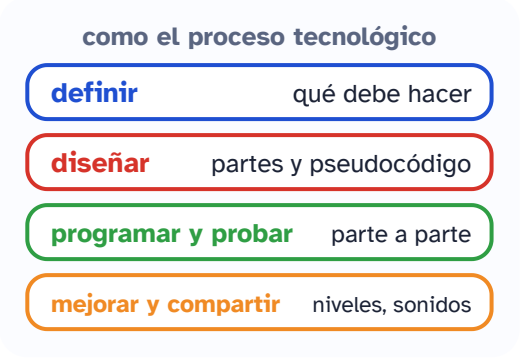

## 3.2 Cómo crear un proyecto de programación

Un proyecto de programación sigue los mismos pasos que el proceso tecnológico. 1) **Definir** qué queremos que haga el programa, con condiciones concretas: «un juego en el que un gato persigue al ratón y gana puntos cada vez que lo toca». 2) **Diseñar**: dividir el problema en partes (los personajes, el movimiento, los puntos, el final) y describir los pasos de cada una, con palabras o con un diagrama, antes de tocar el ordenador. 3) **Programar** cada parte, empezando por la más sencilla. 4) **Probar y depurar** cada parte antes de pasar a la siguiente. 5) **Mejorar**: añadir niveles, sonidos, efectos. 6) **Compartir**: en muchos entornos, como Scratch, se pueden publicar los proyectos para que otros los usen

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

También es importante **guardar a menudo** y con nombres claros, para no perder el trabajo.

Muchos programadores escriben primero el programa en **pseudocódigo**, es decir, con frases sencillas en su idioma: «al empezar, poner puntos a 0; por siempre: si toca al ratón, sumar 1 a puntos y mover el ratón a un lugar al azar». Luego solo hay que traducir cada frase a bloques. Así se piensa la lógica sin preocuparse aún por dónde está cada bloque.



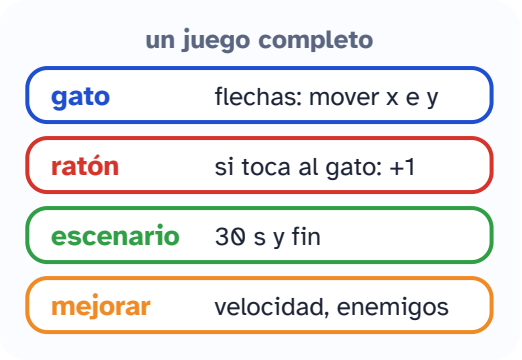

## 3.3 Un juego paso a paso

Veamos cómo se construye un juego sencillo en Scratch: **el gato atrapa ratones**. El jugador mueve al gato con las flechas y debe tocar al ratón, que aparece en sitios al azar. Tiene 30 segundos.

1. **Objetos**: el gato y un ratón. Un fondo de cocina.
2. **Mover al gato**: «al pulsar la tecla flecha derecha → sumar 10 a x»; lo mismo con las otras tres flechas (restar 10 a x, sumar 10 a y, restar 10 a y).
3. **El ratón**: «al hacer clic en la bandera → ir a posición al azar».
4. **Los puntos**: en el ratón, «por siempre: si tocando al gato entonces sumar 1 a puntos, tocar un sonido e ir a posición al azar».
5. **El tiempo**: en el escenario, «al hacer clic en la bandera → fijar puntos a 0, reiniciar el cronómetro, esperar hasta que cronómetro > 30, decir puntos y detener todo».
6. **Probar**: jugar varias partidas y corregir lo que falle. ¿Los puntos vuelven a 0? ¿El ratón aparece fuera del escenario?
7. **Mejorar**: que el ratón se mueva solo, que vaya más rápido cada 5 puntos,

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

Fíjate en que el juego usa todo lo aprendido: eventos, secuencias, bucles, condiciones, variables, operadores y sensores.




### 3.4 Programar objetos del mundo real

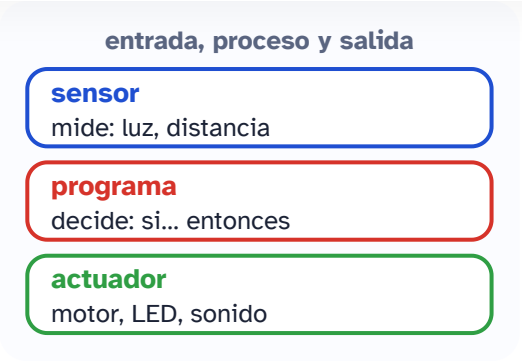
La programación por bloques no sirve solo para juegos en la pantalla. Con ella también se controlan **objetos físicos** que leen el entorno con **sensores** y actúan con **actuadores** (motores, luces, altavoces). Este es el camino hacia la robótica.

- La **micro:bit** se puede convertir en podómetro, brújula, termómetro o en un mando que envía mensajes por radio a otra placa.
- Las placas tipo **Arduino** se programan con entornos de bloques como mBlock o ArduinoBlocks y controlan LED, motores, zumbadores y sensores de luz, distancia o humedad.
- Los **robots educativos** con ruedas siguen una línea negra en el suelo o evitan obstáculos con un sensor de ultrasonidos.

Todos siguen el mismo esquema que ya conoces: **entrada** (el sensor mide algo), **proceso** (el programa decide con una condición) y **salida** (el actuador hace algo). Por ejemplo, una planta inteligente: si la humedad de la tierra es menor que 30, enciende la bomba de agua 3 segundos.

Muchos aparatos que usas cada día funcionan así: la puerta automática del súper, la luz del pasillo que se enciende al pasar o el sensor de aparcamiento de un coche.

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información




## La programación por bloques en ocho ideas




Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

## Ejercicios

1. **¿Qué es un programa?**

2. **Escribe tres ventajas de los lenguajes por bloques.**

3. **Nombra tres entornos de programación por bloques.**

4. **¿Para qué sirven los bloques de eventos?**

5. **¿Qué es un bucle? Pon un ejemplo.**

6. **¿Qué es una condición?**

7. **¿Qué es una variable? Pon un ejemplo.**

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

9. **Escribe tres consejos para depurar.**

10. **Escribe los pasos de un proyecto de programación.**

11. **¿Qué es un algoritmo? Pon un ejemplo de la vida diaria.**

12. **¿Qué forma tiene en un diagrama de flujo una pregunta? ¿Y una acción?**

13. **¿Qué es un objeto (sprite) en Scratch?**

14. **¿Para qué sirven los disfraces?**

15. **¿Qué es un sensor? ¿Y un actuador?**

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

16. **¿Por qué «echa un poco de agua» no es una buena instrucción para un robot?**

17. **En Scratch, ¿dónde está el objeto que está en la posición x = 0, y = 0?**

18. **¿Qué bloque usarías para que el gato se mueva hacia arriba al pulsar la flecha arriba?**

19. **¿Qué diferencia hay entre los operadores lógicos «y» y «o»?**

20. **Escribe en pseudocódigo un programa que haga que el ratón vaya a un sitio al azar cada vez que lo toca el gato.**

21. **Explica qué es la técnica del pato de goma.**

22. **¿Qué es el pensamiento computacional?**

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

23. **Una puerta automática se abre cuando alguien se acerca. Identifica entrada, proceso y salida.**

24. **Dibuja el diagrama de flujo para decidir si salir con paraguas.**

25. **Programa (con bloques o pseudocódigo) una micro:bit que muestre una cara feliz si la temperatura está entre 18 y 25 °C y una triste si no.**

26. **En la lectura, ¿por qué el programa de Marc dejó de funcionar al mover una pared? ¿Cómo lo arreglarías?**

27. **Propón tres mejoras para el juego del gato y el ratón e indica qué bloques necesita cada una.**

28. **Diseña el algoritmo de una planta inteligente con un sensor de humedad y una bomba de agua.**

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

30. **¿Quién creó Scratch y cuánto cuesta?**

31. **Escribe un algoritmo para que un robot salga de cualquier laberinto siguiendo la pared de la derecha.**

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

## Practica

1. **Los bloques que inician un programa son de...**
   - **A)** eventos
   - **B)** operadores
   - **C)** apariencia
   - **D)** sonido
2. **La placa que se programa con MakeCode es la...**
   - **A)** placa base
   - **B)** tarjeta gráfica
   - **C)** memoria RAM
   - **D)** micro:bit
3. **Un bucle sirve para...**
   - **A)** decidir
   - **B)** guardar
   - **C)** repetir
   - **D)** borrar
4. **«Si... entonces» es una...**
   - **A)** variable
   - **B)** condición
   - **C)** secuencia
   - **D)** bucle
5. **Los puntos de un juego se guardan en una...**
   - **A)** condición
   - **B)** variable
   - **C)** secuencia
   - **D)** bucle
6. **Un error en un programa se llama...**
   - **A)** bug
   - **B)** bit
   - **C)** byte

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

**A)** 45°
**B)** 120°
**C)** 180°
**D)** 90°

8. **En una secuencia las instrucciones van...**

**A)** todas a la vez
**B)** al azar
**C)** una tras otra
**D)** de abajo arriba

9. **Los bloques sensores sirven para...**

**A)** repetir
**B)** dibujar
**C)** leer el entorno
**D)** borrar el programa

10. **Scratch sirve sobre todo para crear...**

**A)** hojas de cálculo
**B)** juegos y animaciones
**C)** sistemas operativos
**D)** virus

11. **Una serie de pasos ordenados para resolver un problema es un...**

**A)** algoritmo
**B)** sensor
**C)** disfraz
**D)** píxel

12. **En un diagrama de flujo, la pregunta se dibuja con un...**

**A)** óvalo
**B)** rectángulo
**C)** círculo
**D)** rombo

13. **El inicio y el fin de un diagrama se dibujan con un...**

**A)** rombo

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

**D)** óvalo

14. **En Scratch, los personajes se llaman...**

    **A)** fondos

    **B)** variables

    **C)** objetos (sprites)

    **D)** bucles

15. **El centro del escenario de Scratch es el punto...**

    **A)** (240, 180)

    **B)** (0, 0)

    **C)** (1, 1)

    **D)** (100, 100)

16. **Al empezar un juego, los puntos se...**

    **A)** fijan a 0

    **B)** borran del programa

    **C)** multiplican

    **D)** esconden

17. **El operador «y» exige que...**

    **A)** se cumplan las dos condiciones

    **B)** se cumpla una

    **C)** no se cumpla ninguna

    **D)** se repita

18. **Un sensor de distancia es un elemento de...**

    **A)** salida

    **B)** proceso

    **C)** almacenamiento

    **D)** entrada

19. **Un motor de un robot es un...**

    **A)** sensor

    **B)** operador

    **C)** actuador

    **D)** evento

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

**A)** un disfraz

**B)** pseudocódigo

**C)** un bug

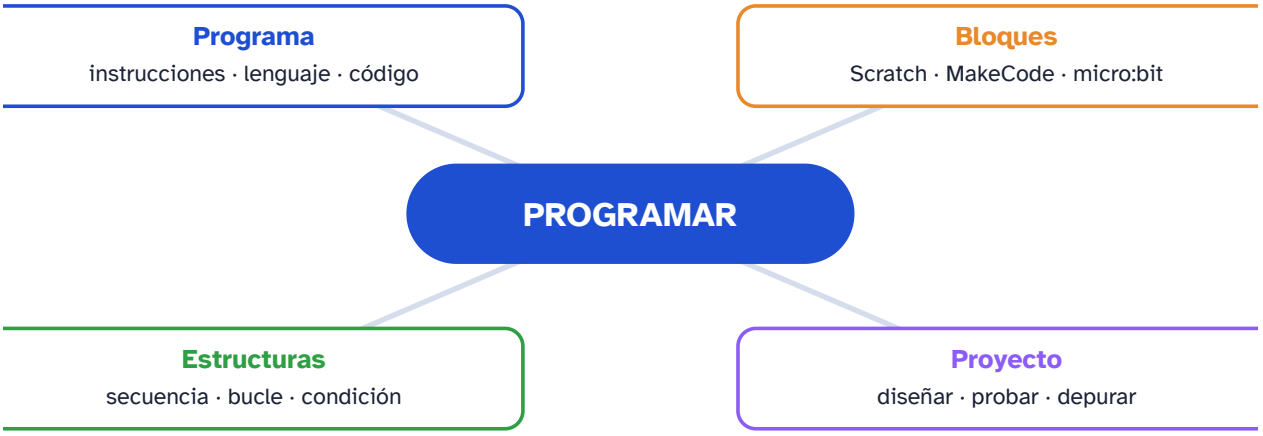
**D)** un fondo

## Explícalo con tus palabras

Incluye: Qué es un programa y por qué una máquina necesita instrucciones precisas; Qué son la secuencia, el bucle y la condición; Cómo se busca un error en un programa.

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

## Resumen




## Glosario

| | |
|---|---|
| **programa** | Lista de instrucciones que dice a una máquina lo que tiene que hacer. |
| **algoritmo** | Serie de pasos ordenados y precisos para resolver un problema. |
| **diagrama de flujo** | Dibujo de un algoritmo con formas y flechas. |
| **lenguaje de programación** | Lenguaje con reglas precisas para escribir programas. |
| **código** | Texto o bloques de un programa. |
| **objeto (sprite)** | Personaje o elemento de un proyecto de Scratch. |
| **evento** | Algo que ocurre y pone en marcha un programa. |
| **secuencia** | Instrucciones que se ejecutan una tras otra. |
| **bucle** | Grupo de instrucciones que se repite. |
| **condición** | Instrucción que solo se ejecuta si se cumple algo. |
| **variable** | Dato con nombre que puede cambiar, como los puntos. |
| **operador** | Bloque que hace cálculos o comparaciones. |
| **depurar** | Buscar y corregir los errores de un programa. |
| **pseudocódigo** | Programa escrito con frases sencillas antes de |

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

## Tarjetas de repaso

Recórtalas: pregunta delante, respuesta detrás.

**Programa**

Lista ordenada de instrucciones para una máquina.

**Lenguaje de programación**

Lenguaje con reglas precisas para escribir programas.

**Evento**

Algo que ocurre e inicia un programa.

**Secuencia**

Instrucciones que se ejecutan una tras otra.

**Bucle**

Repetición de un grupo de instrucciones.

**Condición**

Instrucciones que solo se ejecutan si se cumple algo.

**Variable**

Espacio que guarda un valor que cambia.

**Depurar**

Buscar y corregir errores.

**Algoritmo**

Pasos ordenados, precisos y finitos para resolver un problema.

**Diagrama de flujo**

Dibujo de un algoritmo: óvalo, rectángulo, rombo y flechas.

**Operador**

Bloque que calcula o compara: +, >, y, o, no.

**Sensor y actuador**

El sensor mide el entorno; el actuador hace algo.

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

## Soluciones

1. **A)** Eventos.
2. **D)** micro:bit.
3. **C)** Repetir.
4. **B)** Condición.
5. **B)** Variable.
6. **A)** Bug.
7. **D)** 90°.
8. **C)** En orden.
9. **C)** Leer datos.
10. **B)** Juegos, historias, animaciones.
11. **A)** Como una receta.
12. **D)** Tiene dos salidas.
13. **D)** Forma redondeada.
14. **C)** Cada uno con sus programas.
15. **B)** x = 0, y = 0.
16. **A)** Hay que reiniciar.
17. **A)** Ambas verdaderas.
18. **D)** Mide el entorno.
19. **C)** Hace algo.
20. **B)** Lógica antes que bloques.

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

## Soluciones de los ejercicios

1. Una lista de instrucciones ordenadas que dice a una máquina lo que tiene que hacer.
2. No hay errores de escritura, los bloques solo encajan donde tienen sentido y se ve el resultado al momento.
3. Scratch, MakeCode y App Inventor.
4. Para iniciar el programa cuando ocurre algo.
5. Una repetición de instrucciones: repetir 4 veces avanzar y girar 90° para hacer un cuadrado.
6. Una estructura que ejecuta instrucciones solo si se cumple algo: si… entonces.
7. Un espacio que guarda un valor que puede cambiar, como los puntos de un juego.
8. Buscar y corregir los errores de un programa.
9. Ejecutar paso a paso, probar una parte cada vez y cambiar una sola cosa cada vez.
10. Definir, diseñar, programar, probar y depurar, mejorar y compartir.
11. Una serie de **pasos ordenados y precisos** para resolver un problema. Ejemplo: una **receta** de cocina.
12. La pregunta, un **rombo**; la acción, un **rectángulo**.
13. Un **personaje** o elemento que se mueve en el escenario y tiene sus propios programas.
14. Son dibujos distintos de un objeto; cambiándolos se consigue que parezca que se **mueve** o cambia.
15. El **sensor** mide algo del entorno (luz, distancia); el **actuador** hace algo (motor, luz, sonido).
16. Porque no es **precisa**: el robot necesita una cantidad exacta, como «echa 100 ml de agua».
17. En el **centro** del escenario.
18. «al pulsar la tecla flecha arriba» → «**sumar 10 a y**».
19. Con «**y**» se tienen que cumplir las dos condiciones; con «**o**» basta con una.
20. «Por siempre: si tocando al gato entonces sumar 1 a puntos e ir a posición al azar.»
21. Explicar el programa en **voz alta**, bloque a bloque, para descubrir el error al razonarlo.
22. Forma de pensar que **descompone** un problema en partes, busca patrones y diseña pasos que pueda ejecutar una máquina.
23. **Entrada**: el sensor detecta a la persona. **Proceso**: el programa decide abrir. **Salida**: el motor abre la puerta.
24. Inicio → ¿llueve o el cielo está muy nublado? Si sí: coger el paraguas. Si no: no cogerlo → salir de casa → fin. (Un rombo con dos salidas que se juntan.)
25. «Por siempre: si (temperatura > 17) **y** (temperatura < 26) entonces mostrar cara feliz; si no, mostrar cara triste.»
26. Porque era una **secuencia fija** pensada para un laberinto concreto. Se arregla con un **sensor de distancia** y una condición: «por siempre: si distancia < 10 cm entonces girar;

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información

vidas (nuevo objeto + **variable** vidas).

28. «Por siempre: leer humedad; si humedad < 30 entonces encender bomba 3 segundos y apagarla; esperar 1 hora.»
29. Porque si no, los **bucles «por siempre»** siguen funcionando y el jugador puede seguir sumando puntos después del final.
30. Lo creó el **MIT** y es **gratuito**.
31. «Por siempre: si no hay pared a la derecha, girar a la derecha y avanzar; si no, si no hay pared delante, avanzar; si no, girar a la izquierda.»

Usamos Google Analytics para saber cuántas personas visitan Escolapedia. Solo se activa si lo aceptas. Más información